جزوه جامع و بی‌نظیر فصل دوم (بخش سوم: انواع فراخوانی‌های سیستمی و برنامه‌های سیستمی)

این جزوه بر اساس بخش سوم از فصل دوم فایل ارسالی شما و با هدف پوشش **کامل، عمیق و مفهومی** تمام مطالب، تعاریف فنی، دسته‌بندی‌ها و مثال‌های ذکر شده تهیه شده است. هدف، درک کامل مفاهیم برای کسب بالاترین نمره است.

۲-۳: فراخوانی‌های سیستمی (System Calls) - ادامه

پس از درک مفهوم و چرایی فراخوانی‌های سیستمی، اکنون به بررسی جزئیات پیاده‌سازی و انواع آن‌ها می‌پردازیم.

۲-۳-۲: رابط برنامه‌نویسی کاربردی (Application Programming Interface - API)

با وجود اینکه برنامه‌ها برای انجام کارهای خود به شدت به فراخوانی‌های سیستمی وابسته‌اند، برنامه‌نویسان به ندرت به طور **مستقیم** با آن‌ها کار می‌کنند. در عوض، آن‌ها از یک لایه انتزاعی بالاتر به نام **API** استفاده می‌کنند.

- **API چیست؟**API مجموعه‌ای از **توابع** از پیش تعریف شده است که برنامه‌نویس می‌تواند از آن‌ها در کد خود استفاده کند. API مشخص می‌کند که هر تابع چه پارامترهایی دریافت می‌کند و چه مقداری را بازمی‌گرداند.
- **رابطه API و فراخوانی سیستمی:**توابع موجود در API، در پشت صحنه، فراخوانی‌های سیستمی واقعی و پیچیده‌تر را برای برنامه‌نویس **احضار (Invoke)** می‌کنند. به عبارت دیگر، API یک **پوشش (Wrapper)** کاربرپسند برای فراخوانی‌های سیستمی سطح پایین است.
    - **چرا از API استفاده می‌کنیم؟**

1. **قابلیت حمل (Portability):** برنامه‌ای که با استفاده از یک API استاندارد (مانند **POSIX API** برای سیستم‌های یونیکسی یا **Win32 API** برای ویندوز) نوشته شده باشد، می‌تواند بدون تغییر (یا با تغییرات اندک) روی هر سیستم‌عاملی که از آن API پشتیبانی می‌کند، کامپایل و اجرا شود.
2. **سهولت استفاده (Ease of Use):** کار با توابع API بسیار ساده‌تر از کار با فراخوانی‌های سیستمی خام است که جزئیات فنی و سطح پایینی دارند.

۲-۳-۳: محیط زمان اجرا (Run-Time Environment - RTE)

RTE یک لایه نرم‌افزاری دیگر است که به عنوان واسطه بین API و سیستم‌عامل عمل می‌کند.

- **وظیفه:** RTE مسئولیت مدیریت اجرای یک برنامه را بر عهده دارد و شامل کامپایلر، کتابخانه‌ها و مهم‌تر از همه، **رابط فراخوانی سیستمی** است.
- **رابط فراخوانی سیستمی:**این رابط، فراخوانی تابع API از طرف برنامه را دریافت کرده و آن را به فراخوانی سیستمی مشخصی که هسته سیستم‌عامل می‌فهمد، ترجمه می‌کند.
    - **فرآیند:**

1. به هر فراخوانی سیستمی در هسته، یک **شماره منحصربه‌فرد** اختصاص داده می‌شود.
2. رابط فراخوانی سیستمی، جدولی از این شماره‌ها را نگهداری می‌کند.
3. با فراخوانی یک تابع API، این رابط شماره مربوطه را پیدا کرده، پارامترها را در فرمت مناسب آماده می‌کند و یک **تله (Trap)** به هسته ارسال می‌کند تا فراخوانی سیستمی مورد نظر اجرا شود.

۲-۳-۴ : مکانیزم ارسال پارامتر به هسته

وقتی یک برنامه فراخوانی سیستمی را صدا می‌زند، باید پارامترهای لازم را نیز به هسته ارسال کند (مثلاً نام فایل برای `open`). سه روش اصلی برای این کار وجود دارد:

1. **از طریق ثبات‌ها (Registers):** ساده‌ترین و سریع‌ترین روش. پارامترها مستقیماً در ثبات‌های CPU قرار داده می‌شوند. **محدودیت:** تعداد ثبات‌ها محدود است و برای تعداد زیاد پارامترها مناسب نیست.
2. **از طریق یک بلاک حافظه:** پارامترها در یک بلاک یا جدول در حافظه برنامه قرار داده می‌شوند و **آدرس** آن بلاک از طریق یک ثبات به هسته ارسال می‌شود. این روش محدودیتی در تعداد پارامترها ندارد. (لینوکس برای بیش از ۵ پارامتر از این روش استفاده می‌کند). 3. **از طریق پشته (Stack):** پارامترها در پشته برنامه قرار داده (Push) می‌شوند و هسته آن‌ها را از پشته برمی‌دارد (Pop). این روش نیز محدودیتی در تعداد پارامترها ندارد.

۲-۴ : انواع فراخوانی‌های سیستمی

فراخوانی‌های سیستمی را می‌توان بر اساس کارکردشان در شش دسته اصلی طبقه‌بندی کرد:

1. **کنترل پردازه (Process Control):**
   - `create_process`, `terminate_process` (در یونیکس: `fork()`, `exit()`): برای ایجاد و خاتمه دادن به پردازه‌ها.
   - `load`, `execute` (در یونیکس: `exec()`): برای بارگذاری و اجرای یک برنامه جدید.
   - `wait_event`, `signal_event` (در یونیکس: `wait()`): برای همگام‌سازی و منتظر ماندن برای وقوع یک رویداد.
   - `allocate_memory`, `free_memory`: برای تخصیص و آزادسازی حافظه.
1. **مدیریت فایل (File Management):**
   - `create_file`, `delete_file`: برای ایجاد و حذف فایل‌ها.
   - `open`, `close`: برای باز کردن و بستن فایل‌ها.
   - `read`, `write`, `reposition`: برای خواندن، نوشتن و جابجایی در فایل.
   - `get_file_attributes`, `set_file_attributes`: برای خواندن و تنظیم ویژگی‌های فایل (مانند مجوزها).
1. **مدیریت دستگاه (Device Management):**
   - `request_device`, `release_device`: برای درخواست دسترسی انحصاری به یک دستگاه و آزاد کردن آن.
   - `read`, `write`, `reposition`: مشابه عملیات فایل، زیرا بسیاری از سیستم‌عامل‌ها (مانند یونیکس) با دستگاه‌ها مانند فایل رفتار می‌کنند.
1. **نگهداری اطلاعات (Information Maintenance):**
   - `get_time_or_date`, `set_time_or_date`: برای دسترسی به ساعت و تاریخ سیستم.
   - `get_system_data`: برای دریافت اطلاعات کلی سیستم (مانند نسخه سیستم‌عامل، میزان حافظه آزاد).
   - `get_process_attributes`, `set_process_attributes`: برای خواندن و تنظیم ویژگی‌های یک پردازه (مانند اولویت اجرا).
   - **عیب‌یابی (Debugging):** فراخوانی‌هایی برای گرفتن **رونوشت از حافظه (Memory Dump)** یا اجرای برنامه به صورت **گام به گام (Single Step)**.
1. **ارتباطات (Communications):**

- :create_communication_connection, delete_communication_connection
  برای برقراری و قطع یک ارتباط (مثلاً در شبکه).
- **مدل عبور پیغام:** send_message, receive_message.
- **مدل حافظه مشترک:** shared_memory_create, shared_memory_attach.

1. **حفاظت (Protection):**
   - get_file_permissions, set_file_permissions (در یونیکس: chmod()): برای کنترل دسترسی به فایل‌ها.
   - allow_user, deny_user.

۲-۵: برنامه‌های سیستمی (System Programs)

برنامه‌های سیستمی، که به آن‌ها **ابزارهای سودمند سیستمی (System Utilities)** نیز گفته می‌شود، یک لایه بالاتر از فراخوانی‌های سیستمی قرار دارند و محیطی راحت‌تر برای توسعه و اجرای برنامه‌ها فراهم می‌کنند.

- **تفاوت با برنامه‌های کاربردی:** خط مرز مشخصی وجود ندارد، اما برنامه‌های سیستمی معمولاً با خود سیستم‌عامل عرضه می‌شوند و برای مدیریت سیستم به کار می‌روند، در حالی که برنامه‌های کاربردی برای حل مسائل کاربران نهایی طراحی شده‌اند.
- **تفاوت با فراخوانی‌های سیستمی:** بسیاری از برنامه‌های سیستمی، در واقع پوشش‌های ساده‌ای برای فراخوانی‌های سیستمی هستند (مثلاً دستور mkdir در لینوکس، فراخوانی سیستمی mkdir() را صدا می‌زند).

دسته‌بندی برنامه‌های سیستمی:

1. **مدیریت فایل:** برنامه‌هایی برای ایجاد، حذف، کپی، تغییر نام و نمایش فایل‌ها و دایرکتوری‌ها.
2. **اطلاعات وضعیت:** ابزارهایی برای نمایش اطلاعات سیستم (مانند تاریخ، میزان حافظه، فضای دیسک، اطلاعات کاربران وارد شده).
3. **ویرایش فایل:** ویرایشگرهای متنی (مانند vi یا nano).
4. **پشتیبانی از زبان‌های برنامه‌نویسی:** کامپایلرها، اسمبلرها، دیباگرها و مفسرها.
5. **بارگذاری و اجرای برنامه: بارگذارها (Loaders)** و **پیونددهنده‌ها (Linkers)** که مسئول آماده‌سازی و اجرای یک برنامه هستند.
6. **ارتباطات:** برنامه‌هایی برای ایجاد ارتباط بین کاربران و سیستم‌های مختلف (مانند ایمیل، مرورگر وب، و ابزارهای انتقال فایل).
7. **برنامه‌های پس‌زمینه (Background Services):** سرویس‌هایی که به طور مداوم در پس‌زمینه اجرا می‌شوند و وظایف مشخصی را انجام می‌دهند (مانند چک کردن وجود دیسک جدید، یا اجرای کارهای زمان‌بندی شده). در یونیکس به آن‌ها **دیمن (Daemon)** و در ویندوز **سرویس (Service)** گفته می‌شود.

//

یعنی میگیم برنامه سیستمی خودش با api ها ساخته شده؟ و وقتی ما عملیاتی انجام میدیم این برنامه سیستمی با استفاده از api های سازنده ان اون عملیات رو به محیط ران تایم میده؟

بله، دقیقاً همینطور است! درک شما الان کاملاً درست و دقیق است.

خلاصه به زبان ساده:

1. برنامه سیستمی (مثل File Explorer) یک نرم‌افزار آماده است که قبلاً توسط برنامه‌نویسان با استفاده از توابع API ساخته شده است.
2. وقتی شما یک عملیات انجام می‌دهید (مثلاً یک فایل را حذف می‌کنید)، در واقع آن بخش از کدِ برنامه سیستمی را فعال می‌کنید.
3. آن کد، تابع مربوطه را از API (مثلاً تابع DeleteFile()) صدا می‌زند.
4. سپس محیط زمان اجرا (Run-Time Environment) این درخواست API را گرفته و آن را برای هسته (Kernel) ترجمه می‌کند.

شما زنجیره را به درستی درک کرده‌اید:

کاربر ⬅ برنامه سیستمی ⬅ API ⬅ محیط زمان اجرا ⬅ فراخوانی سیستمی ⬅ هسته